语音合成

语音合成

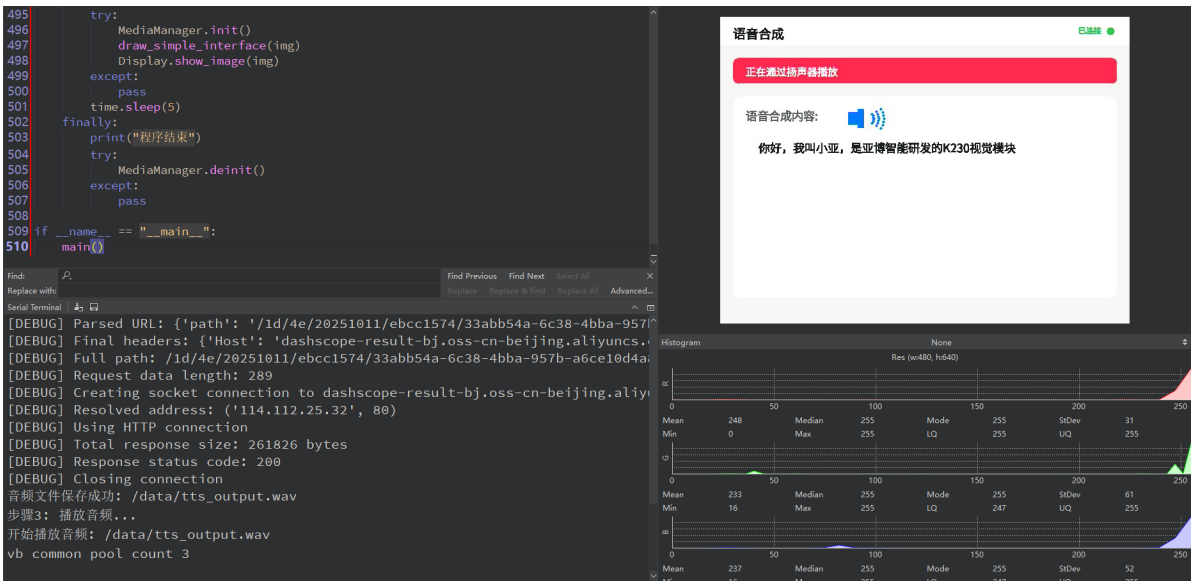
- 例程效果
- 代码讲解
  - 1. 导入必要的模块
  - 2. 定义全局变量和常量
  - 3. 定义 `TTSState` 类
  - 4. 辅助函数
    - 4.1 WiFi连接函数
    - 4.2 UI绘制函数
    - 4.3 语音合成函数
    - 4.4 音频播放函数
  - 5. 主程序逻辑
  - 6. 异常处理与资源释放

例程效果

> 语音合成播放需要用到外接的扬声器，扬声器的具体使用方式请参考前面相关章节的内容

本节例程源码文件名为 `tts_demo.py`。

连接IDE，在IDE中运行代码，你将看到一个语音合成界面。程序会先连接WiFi，然后使用指定的文本（如“你好，我叫小亚...”）通过DashScope TTS API合成语音，并播放音频。界面实时显示状态，如连接中、合成中和播放中，支持动画反馈。



代码讲解

1. 导入必要的模块

```
import uos
import time
import gc
import ujson as json
from media.display import *
from media.media import *
from media.pyaudio import *
import media.wave as wave
from ybUtils.YbSpeaker import YbSpeaker
import YbRequests as requests
```

- `uos`：用于操作系统相关操作，如退出点。
- `time`：用于时间控制和延时。
- `gc`：用于垃圾回收。
- `ujson`：用于JSON数据处理。
- `media.display`，`media.media`，`media.pyaudio`，`media.wave`：用于屏幕显示、媒体管理和音频处理。
- `ybUtils.YbSpeaker`：用于音频播放。
- `YbRequests`：用于发送HTTP请求。

2. 定义全局变量和常量

```
API_KEY = ""
WIFI_SSID = ""
WIFI_KEY = ""
DISPLAY_WIDTH = 640
DISPLAY_HEIGHT = 480
network_connected = False
UI_COLORS = { ... }
TEXT = "你好，我叫小亚，是亚博智能研发的K230视觉模块"
DEFAULT_VOICE = "Cherry"
```

- `API_KEY`，`WIFI_SSID`，`WIFI_KEY`：API密钥和WiFi连接信息（需用户配置）。
- `DISPLAY_WIDTH`，`DISPLAY_HEIGHT`：屏幕分辨率常量。
- `network_connected`：布尔值，记录网络连接状态。
- `UI_COLORS`：字典，定义界面颜色。
- `TEXT`：默认合成文本。
- `DEFAULT_VOICE`：默认语音类型，如"Cherry"。

3. 定义 `TTSState` 类

```
class TTSState:
    def __init__(self):
        self.program_state = "idle"
        self.status_message = "正在初始化..."
        self.audio_file_path = "/data/tts_output.wav"
        self.loading_frame = 0
```

- `TTSState` 类用于管理程序状态，包括当前状态、消息、音频文件路径和加载帧数。

4. 辅助函数

4.1 WiFi连接函数

```
def connect_wifi():
    """连接WiFi网络"""
    global network_connected
    # (函数内容详见代码)
```

- `connect_wifi()`：尝试连接WiFi，支持超时机制，返回连接成功与否。

4.2 UI绘制函数

- `draw_rounded_rect()`：绘制圆角矩形。
- `draw_audio_wave_animation()`：绘制音频波形动画。
- `draw_speaker_icon()`：绘制扬声器图标。
- `draw_loading_spinner()`：绘制加载动画。
- `draw_text_with_wrapping()`：绘制带换行的文本。
- `draw_main_interface()`：绘制主界面，根据状态显示不同元素。

4.3 语音合成函数

```
def synthesize_speech(text, voice="Cherry"):
    # (函数内容详见代码)
```

- `synthesize_speech()`：向DashScope API发送请求合成语音，获取音频URL并下载保存为WAV文件。返回合成成功与消息。

4.4 音频播放函数

```
def play_audio():
    # (函数内容详见代码)
```

- `play_audio()`：播放保存在指定路径的音频文件，使用PyAudio和YbSpeaker。

5. 主程序逻辑

```
def main():
    global state
    Display.init(Display.ST7701, width=DISPLAY_WIDTH, height=DISPLAY_HEIGHT,
to_ide=True)
    MediaManager.init()
    img = image.Image(DISPLAY_WIDTH, DISPLAY_HEIGHT, image.ARGB8888)

    print("=== 亚博智能K230语音合成系统 ===")
    animation_frame = 0

    try:
        state.program_state = "connecting"
        if not connect_wifi():
            state.program_state = "error"
            return

        state.program_state = "synthesizing"
```

```
        success, message = synthesize_speech(TEXT, DEFAULT_VOICE)

        if not success:
            state.program_state = "error"
            return

        state.program_state = "playing"
        success, message = play_audio()

        if not success:
            state.program_state = "error"
            return

        state.program_state = "completed"

        for i in range(100):
            draw_main_interface(img, animation_frame)
            Display.show_image(img)
            time.sleep(0.1)
            animation_frame = (animation_frame + 1) % 100

    except KeyboardInterrupt:
        print("程序被用户中断")
    except Exception as e:
        state.program_state = "error"
        state.status_message = f"程序异常: {str(e)}"
    finally:
        MediaManager.deinit()
        print("程序结束")
```

- 初始化显示和媒体管理。
- 步骤1: 连接WiFi。
- 步骤2: 语音合成。
- 步骤3: 播放音频。
- 循环更新UI动画。
- 异常处理：捕获错误并更新状态。

6. 异常处理与资源释放

- 在 `main()` 函数中使用 `try-except-finally` 块处理异常，确保在程序结束时释放资源，`MediaManager.deinit()`。